

sheLLaMa as a Cloud AI Cost Estimation Platform

Measuring Local LLM Usage to Project Cloud Provider Spend

sheLLaMa Project

May 2026

Abstract

Organizations adopting AI face a critical question: what would our workload cost on cloud AI providers? sheLLaMa answers this by running inference locally on commodity hardware while simultaneously tracking every token consumed and projecting costs across 28+ cloud models from OpenRouter, Amazon Bedrock, OpenAI, Anthropic, Google, xAI, and Meta. This paper describes the architecture, methodology, and practical use of sheLLaMa as a cloud AI cost estimation and capacity planning tool.

Contents

1	Introduction	2
2	The Cost Estimation Problem	2
2.1	Why Estimation is Hard	2
2.2	sheLLaMa’s Approach	2
3	Architecture for Cost Tracking	3
3.1	Token Recording Pipeline	3
3.2	Pricing Data Sources	3
3.3	Cost Calculation	3
4	Supported Cloud Providers and Models	4
4.1	OpenRouter Models (via API)	4
4.2	Amazon Bedrock Models (via AWS Pricing API)	4
4.3	Enterprise Subscriptions (Seat-Based)	4
5	Cost Estimation Interfaces	5
5.1	Admin Console — Costs Page	5
5.2	Admin Console — Cloud Usage Page	5
5.3	CLI Benchmarking	6
5.4	REST API	6
6	Use Cases for Cost Estimation	6
6.1	Pre-Migration Planning	6
6.2	Budget Forecasting	6
6.3	Provider Comparison	7
6.4	ROI Calculation for Local Infrastructure	7
6.5	Hybrid Strategy Optimization	7

7	Implementation Details	8
7.1	Persistent Statistics	8
7.2	Benchmark Exclusion	8
7.3	Cache Awareness	8
7.4	Live Pricing Refresh	8
7.5	Per-Key Budget Enforcement	8
8	Deployment for Cost Estimation	9
8.1	Minimal Setup (Single Server)	9
8.2	Distributed Setup (Production)	9
9	Interpreting Results	9
9.1	Reading the Cost Table	9
9.2	Token Ratio Analysis	10
9.3	Quality vs. Cost Tradeoff	10
9.4	Subscription vs. Pay-Per-Token	10
10	Limitations	11
11	Conclusion	11
12	References	11

1 Introduction

Cloud AI services charge per token—typically split into input (prompt) and output (completion) tokens. Pricing varies by orders of magnitude: from \$0.035/M tokens (Amazon Nova Micro) to \$75/M tokens (Claude Opus 4 output). For organizations evaluating AI adoption, the question is not whether AI is useful, but whether the cost is justified at their usage volume.

sheLLaMa is an AI-powered agentic shell that integrates directly into bash and PowerShell. The AI reads files, writes code, runs commands, searches the web, and iterates—with permission tiers, conversation memory, and context awareness. It supports local LLMs (Ollama), cloud fallback, and distributed backends. Crucially, it also doubles as a cloud AI cost estimation platform:

1. Run your actual AI workloads locally (code generation, chat, analysis, shell automation)
2. The platform counts every prompt and response token consumed
3. Real-time cost projections show what the same usage would cost on each cloud provider
4. Per-client and per-API-key cost attribution enables team-level budgeting
5. Benchmarking compares local model quality against cloud alternatives

The result is a data-driven cost estimate based on *actual usage patterns*, not hypothetical projections.

2 The Cost Estimation Problem

2.1 Why Estimation is Hard

Cloud AI pricing is complex:

- **Asymmetric pricing:** Input and output tokens have different rates (e.g., Claude 4 Sonnet: \$3/M input, \$15/M output)
- **Variable output length:** The same prompt can produce vastly different response lengths depending on the model and task
- **Workload mix:** Organizations use AI for diverse tasks—chat, code generation, document analysis, shell automation—each with different token profiles
- **Rapid price changes:** Cloud providers update pricing frequently; static spreadsheet estimates become stale
- **Hidden costs:** Rate limiting, retries, conversation context accumulation, and cache misses all inflate real-world spend

2.2 sheLLaMa’s Approach

Rather than estimating from assumptions, sheLLaMa measures actual usage:

- Every request records `prompt_tokens` and `response_tokens` from Ollama’s response metadata
- Token counts are accumulated per client IP, per task type, and per API key
- Cost projections use **live pricing** fetched from OpenRouter’s API and AWS Pricing API
- Static fallback prices are maintained for when APIs are unreachable
- Benchmark tokens are excluded from cost projections to reflect real usage only

3 Architecture for Cost Tracking

3.1 Token Recording Pipeline

Every request flowing through the frontend load balancer is instrumented:

Listing 1: Token recording in `proxy_request`

```
record_ip_tokens(
    ip=client_ip,
    tokens=result.get('total_tokens', 0),
    task_type=task_type,
    prompt_tokens=result.get('prompt_tokens', 0),
    response_tokens=result.get('response_tokens', 0),
    cloud_fallback=result.get('cloud_fallback', False),
    key_name=get_key_name()
)
```

Tokens are stored in three dimensions:

- **Cumulative totals** (persisted to disk, survive restarts)
- **Per-IP history** (timestamped entries for time-range queries)
- **Per-task breakdown** (chat, generate, explain, analyze, image)

3.2 Pricing Data Sources

Source	Models Covered	Refresh
OpenRouter API	Claude, GPT-4o, Gemini, Grok, Llama, Nova (15 models)	Per /test request
AWS Pricing API	Bedrock on-demand: Nova, Llama, DeepSeek, Mistral (9 models)	Per /test request
Static fallback	All 28 models (hardcoded)	Code updates

Table 1: Pricing data sources and coverage

The system attempts live pricing first. If OpenRouter or AWS is unreachable (offline environments), static fallback prices are used. The response always includes `pricing_source` (`openrouter` or `static`) so users know the data freshness.

3.3 Cost Calculation

For each provider, the cost formula is:

$$\text{Cost} = \frac{\text{prompt_tokens} \times P_{\text{input}}}{1,000,000} + \frac{\text{response_tokens} \times P_{\text{output}}}{1,000,000}$$

where P_{input} and P_{output} are per-million-token rates.

4 Supported Cloud Providers and Models

4.1 OpenRouter Models (via API)

Model	Input (\$/M)	Output (\$/M)
Claude 4 Sonnet	3.00	15.00
Claude 4 Haiku	1.00	5.00
Claude 3.5 Sonnet	3.00	15.00
GPT-4o	2.50	10.00
GPT-4o mini	0.15	0.60
OpenAI o3	2.00	8.00
OpenAI o4-mini	1.10	4.40
Gemini 2.5 Pro	1.25	10.00
Gemini 2.5 Flash	0.30	2.50
Grok 3	3.00	15.00
Grok 3 mini	0.30	0.50
Llama 3.1 70B	0.40	0.40
Amazon Nova Pro	0.80	3.20
Amazon Nova Lite	0.06	0.24
Amazon Nova Micro	0.04	0.14

Table 2: OpenRouter model pricing (static fallback values; live prices may differ)

4.2 Amazon Bedrock Models (via AWS Pricing API)

Model	Input (\$/M)	Output (\$/M)
Claude Opus 4	15.00	75.00
Claude 4 Sonnet	3.00	15.00
Claude 3.5 Sonnet	3.00	15.00
Claude 3.5 Haiku	0.80	4.00
Nova Pro	0.80	3.20
Nova Lite	0.06	0.24
Nova Micro	0.035	0.14
Nova Premier	2.50	12.50
Llama 4 Maverick	0.24	0.97
Llama 4 Scout	0.17	0.66
Llama 3.3 70B	0.72	0.72
DeepSeek R1	1.35	5.40
Mistral Large 3	0.50	1.50

Table 3: Amazon Bedrock on-demand pricing (us-east-1, static fallback values)

4.3 Enterprise Subscriptions (Seat-Based)

Unlike per-token APIs, enterprise AI services charge a flat monthly fee per user. sheLLaMa compares these by calculating what your actual token usage would cost on the underlying per-token API, then showing the difference against the subscription price.

Service	Underlying Model	\$/seat/month
Microsoft Copilot	GPT-4o	30.00
ChatGPT Team	GPT-4o, o3	30.00
ChatGPT Enterprise	GPT-4o, o3 (unlimited)	60.00
Claude Team	Claude 4 Sonnet, Opus 4	30.00
Claude Enterprise	Claude 4 Sonnet (higher limits)	60.00
Google Gemini Business	Gemini 2.5 Pro	24.00
Google Gemini Enterprise	Gemini 2.5 Pro (unlimited)	36.00
Grok (X Premium+)	Grok 3	40.00
Amazon Q Developer Pro	Bedrock Claude 4 Sonnet	19.00
Amazon Q Business Pro	Bedrock Nova Pro	20.00

Table 4: Enterprise AI subscription pricing (monthly per seat)

The comparison answers: “Given our team’s actual token consumption, is a per-seat subscription cheaper or more expensive than pay-per-token API access?”

For light users, subscriptions are expensive relative to actual API consumption. For heavy users who would exceed API budgets, subscriptions provide cost certainty. The `/enterprise-costs?users=N&` endpoint calculates this dynamically.

5 Cost Estimation Interfaces

5.1 Admin Console — Costs Page

The web-based costs page (`/costs`) provides:

- **Time range filtering:** Day, week, month, year, all-time, or custom date range
- **Hypothetical costs:** What your total local usage would cost on each cloud provider
- **Actual fallback spend:** Real costs incurred when cloud fallback was triggered
- **Cached savings:** Tokens served from cache that would have cost money on cloud
- **Visual indicators:** Cheapest provider highlighted green, most expensive in red
- **Bedrock section:** Separated with AWS orange styling for clarity
- **Live pricing source:** Shows whether prices are from OpenRouter or static fallback

5.2 Admin Console — Cloud Usage Page

The cloud usage page (`/cloud-usage`) provides per-client cost attribution:

- **By Client IP:** Token usage and projected cloud cost per client, with expandable provider breakdown
- **By API Key:** Same breakdown per authenticated API key
- **By Cloud Model:** Which fallback models were used and their token consumption
- **Cost range:** Each row shows cheapest and most expensive provider estimate
- **Expandable detail:** Click to see full provider-by-provider cost table per client

The underlying API endpoint (`/cloud-usage-data`) returns JSON with cost estimates computed using the same pricing pipeline as the costs page.

5.3 CLI Benchmarking

The `,test` command provides per-request cost estimates:

Listing 2: CLI benchmark with cost estimates

```
$ ,test all
Testing 4 models with default prompt...
```

Model	Time	Prompt	Response	Total	tok/s
qwen2.5-coder:3b	12.3s	142	387	529	43.1
qwen2.5-coder:7b	34.7s	142	412	554	16.0
deepseek-coder:6.7b	38.2s	142	395	537	14.1
qwen2.5-coder:14b	97.1s	142	445	587	6.0

Cloud cost estimates (avg 142 prompt + 410 response tokens):

Provider	Input	Output	Total
Claude 4 Sonnet	\$0.000426	\$0.006150	\$0.006576
GPT-4o	\$0.000355	\$0.004100	\$0.004455
Amazon Nova Micro	\$0.000005	\$0.000057	\$0.000062
...			

5.4 REST API

Listing 3: Cost estimation via API

```
# Get cumulative cost projections
curl http://server:5000/cloud-costs

# Get costs for a specific time range
curl "http://server:5000/cost-history?since=1715900000&until=1716000000"

# Benchmark with cost comparison
curl -X POST http://server:5000/test \
  -H "Content-Type: application/json" \
  -d '{"model": "all"}'
```

6 Use Cases for Cost Estimation

6.1 Pre-Migration Planning

Before committing to a cloud AI provider, run sheLLaMa for 1–4 weeks with your team’s actual workload. The costs page will show:

- Total monthly projected spend per provider
- Which provider offers the best price for your workload mix
- Whether cheaper models (Nova Micro, GPT-4o mini) are viable alternatives
- The ratio of input to output tokens in your usage (affects provider choice)

6.2 Budget Forecasting

The time-range filtering enables trend analysis:

- Daily token consumption patterns (peak hours, quiet periods)
- Week-over-week growth as AI adoption increases
- Per-user or per-team breakdown via API key tracking
- Seasonal patterns in development cycles

6.3 Provider Comparison

The 28-model comparison table answers questions like:

- “Is Bedrock cheaper than OpenRouter for our workload?”
- “How much would we save switching from Claude to Nova Pro?”
- “What’s the cost difference between GPT-4o and GPT-4o mini for our use case?”
- “Is the 5x price premium of Claude Opus justified over Claude Sonnet?”

6.4 ROI Calculation for Local Infrastructure

Compare the projected cloud spend against the cost of local hardware:

Scenario	Local Cost	Cloud Cost (monthly)
Light usage (10K tokens/day)	Server: \$2,000 one-time	\$0.50–\$15/mo
Medium usage (100K tokens/day)	Server: \$4,000 one-time	\$5–\$150/mo
Heavy usage (1M tokens/day)	Server: \$8,000 one-time	\$50–\$1,500/mo
Team (10 devs, 5M tokens/day)	2 servers: \$10,000	\$250–\$7,500/mo

Table 5: Break-even analysis: local hardware vs. cloud AI spend

At heavy usage, local infrastructure pays for itself in 1–6 months depending on the cloud provider chosen.

6.5 Hybrid Strategy Optimization

sheLLaMa’s cloud fallback feature enables a hybrid approach:

1. Run most requests locally (zero marginal cost)
2. Fall back to cloud only when local quality is insufficient
3. Track actual fallback spend separately from hypothetical costs
4. Set per-key budgets to cap cloud spend: `"budget": {"max_daily": 5.00}`
5. Receive webhook alerts at 80% budget consumption

The costs page separates hypothetical costs (“what if everything ran on cloud”) from actual fallback spend (“what we actually paid”), giving a clear picture of savings.

7 Implementation Details

7.1 Persistent Statistics

Cost data survives service restarts:

- Cumulative token totals saved to `persisted_totals.json` every 60 seconds
- Per-IP history maintained in memory with configurable retention
- Background thread handles periodic persistence
- `/reset-cloud-costs` endpoint for admin to clear data (requires admin role)

7.2 Benchmark Exclusion

Tokens from `/test` benchmark requests are excluded from cost projections. This ensures the running tab reflects real usage only—not artificial load from model comparison tests.

7.3 Cache Awareness

When prompt caching is enabled (default 5-minute TTL), cached responses are tracked separately. The costs page shows how many tokens were served from cache—representing additional savings that would not exist on most cloud providers (which charge per request regardless of caching).

7.4 Live Pricing Refresh

Listing 4: Live pricing fetch from OpenRouter

```
def fetch_cloud_pricing():
    resp = requests.get('https://openrouter.ai/api/v1/models', timeout=10)
    data = resp.json().get('data', [])
    by_id = {}
    for m in data:
        p = m.get('pricing', {})
        prompt = float(p.get('prompt', '0'))
        comp = float(p.get('completion', '0'))
        if prompt > 0 or comp > 0:
            by_id[m['id']] = {'input': prompt * 1e6, 'output': comp * 1e6}
    # Map display names to live prices...
```

Bedrock pricing is fetched via the AWS CLI:

Listing 5: AWS Pricing API call

```
aws pricing get-products --service-code AmazonBedrock \
  --region us-east-1 \
  --filters Type=TERM_MATCH,Field=regionCode,Value=us-east-1
```

7.5 Per-Key Budget Enforcement

API keys can have spending limits that are enforced on actual cloud fallback spend:

Listing 6: API key with budget limit

```
{
```

```

"api_keys": {
  "sk-team-dev": {
    "name": "dev-team",
    "role": "user",
    "budget": {"max_daily": 10.00},
    "rate_limit": {"rpm": 30, "tpd": 500000}
  }
}
}

```

When the budget is 80% consumed, a webhook fires. At 100%, cloud fallback requests are rejected with HTTP 429.

8 Deployment for Cost Estimation

8.1 Minimal Setup (Single Server)

Listing 7: Quick start for cost estimation

```

# Install and start
ollama pull qwen2.5-coder:7b
pip install flask ollama pyyaml requests psutil
python backend/app.py

# Point your team's CLI clients at the server
export SHELLAMA_API=http://server:5000
export SHELLAMA_MODEL=qwen2.5-coder:7b
source cli/shellama.bash

# Use AI normally for 1-4 weeks, then check costs
curl http://server:5000/cloud-costs | python -m json.tool

```

8.2 Distributed Setup (Production)

For larger teams, deploy the frontend load balancer with multiple backends:

Listing 8: backends.json for multi-server deployment

```

{
  "backends": [
    {"url": "http://192.168.1.230:5000", "weight": 10, "max_model":
      "qwen2.5-coder:7b"},
    {"url": "http://192.168.1.233:5000", "weight": 10, "max_model":
      "qwen2.5-coder:14b"}
  ]
}

```

All token tracking is centralized on the frontend, regardless of which backend processes the request.

9 Interpreting Results

9.1 Reading the Cost Table

A typical `/cloud-costs` response after one week of team usage:

```
{
  "prompt_tokens": 847293,
  "response_tokens": 2341567,
  "total_tokens": 3188860,
  "pricing_source": "openrouter",
  "cloud_costs": [
    {"provider": "Claude 4 Sonnet", "total_cost": 37.66},
    {"provider": "GPT-4o", "total_cost": 25.53},
    {"provider": "Gemini 2.5 Flash", "total_cost": 6.11},
    {"provider": "Amazon Nova Micro", "total_cost": 0.36},
    {"provider": "Bedrock Claude Opus 4", "total_cost": 188.22}
  ]
}
```

Interpretation: This team's weekly AI usage would cost between \$0.36 (Nova Micro) and \$188.22 (Claude Opus 4) on cloud providers. The local infrastructure cost is \$0 marginal after hardware purchase.

9.2 Token Ratio Analysis

The ratio of prompt to response tokens affects provider choice:

- **High output ratio** (code generation, explanations): Providers with low output pricing win (Nova Micro, Llama, GPT-4o mini)
- **High input ratio** (analysis of large files, context-heavy chat): Providers with low input pricing win (Gemini with large context, Nova Lite)
- **Balanced:** Overall total cost is the deciding factor

9.3 Quality vs. Cost Tradeoff

The `,test` benchmark lets you compare local model quality against cloud alternatives. If a \$0.04/request local model produces acceptable output for 95% of tasks, the ROI of local infrastructure is clear. The 5% that needs cloud fallback can be budgeted precisely using the actual fallback tracking.

9.4 Subscription vs. Pay-Per-Token

The enterprise comparison reveals whether seat-based subscriptions make financial sense:

Listing 9: Enterprise cost comparison API

```
# Compare for a 10-person team over 30 days of tracked usage
curl "http://server:5000/enterprise-costs?users=10&days=30"
```

```
{
  "enterprise_costs": [
    {"provider": "Microsoft Copilot", "monthly_total": 300.00,
     "equivalent_token_cost": 2.25, "savings_vs_subscription": 297.75},
    {"provider": "Amazon Q Developer Pro", "monthly_total": 190.00,
     "equivalent_token_cost": 1.80, "savings_vs_subscription": 188.20}
  ]
}
```

Interpretation: If the subscription costs \$300/month but the equivalent API usage is only \$2.25, the team is paying 133x more for the subscription than they would on per-token pricing. This is common for light-to-moderate usage. Heavy users (millions of tokens/day) may find subscriptions cheaper due to unlimited usage caps.

10 Limitations

- **Token counts are model-dependent:** Different tokenizers produce different counts for the same text. Local Ollama token counts are approximate proxies for cloud provider counts.
- **Pricing changes:** Live pricing is fetched per-request for benchmarks but cached otherwise. Prices may change between fetches.
- **No latency costing:** Cloud providers may offer faster responses; the cost of developer wait time is not captured.
- **No quality weighting:** A cheap model that requires 3 retries may cost more than an expensive model that succeeds first try. This is partially addressed by the fallback tracking.
- **Region-specific:** Bedrock pricing is for us-east-1. Other regions may differ.

11 Conclusion

sheLLaMa transforms local LLM infrastructure into a cloud AI cost estimation platform. By running actual workloads locally and projecting costs across 28+ cloud models with live pricing, organizations can make data-driven decisions about:

- Whether to adopt cloud AI services
- Which provider offers the best value for their specific workload
- When local infrastructure ROI exceeds cloud spend
- How to budget for hybrid local+cloud deployments
- Per-team and per-project cost allocation

The platform requires no cloud accounts, no API keys, and no ongoing costs to operate as a cost estimator. It runs fully offline after the initial model pull, making it suitable for air-gapped environments and organizations evaluating AI adoption without committing to vendor contracts.

12 References

- OpenRouter pricing API: <https://openrouter.ai/api/v1/models>
- AWS Bedrock pricing: <https://aws.amazon.com/bedrock/pricing/>
- Ollama: <https://ollama.com>
- sheLLaMa source: <https://github.com/rory/shellama>